Technical report: Propaganda Radar

Pro-government outlets dominate the Serbian media landscape and systematically push narratives that favour the ruling party [1]. These outlets often wage smear campaigns against actors deemed hostile to the authorities [2]. On the other hand, their coverage of the authorities is overwhelmingly positive.

The CRTA watchdog organisation monitors the front pages of pro-government media for such manipulations and publishes monthly media manipulation reports [3]. CRTA identifies four categories of media manipulation in Serbia:

1. Vilifying opponents: Coverage shaming the student movement, portraying them as criminals or as wanting to destroy Serbia
2. Vilifying neighbours: Coverage painting neighbouring countries and their leaders as part of a conspiracy against Serbia
3. Personality cult: Coverage praising former president Aleksandar Vučić, framing him as an indispensable political mastermind and his achievements as historic
4. Vilifying the EU: Anti-EU content portraying it as unfriendly to Serbia and opposed to its interests.

While these reports successfully identify patterns in Serbian pro-regime media, they are published too infrequently, and no report has been published since August [3]. Moreover, the headlines appear to be hand-labelled, which is labour-intensive and time-consuming. Additionally, coverage is limited to the front pages and does not extend to the inner pages.

This project aims to develop a web service that automatically identifies and categorises headlines on pro-regime outlet websites using classification algorithms. This approach would enable real-time narrative monitoring and expand the scope of analysis. However, because machine learning models may incorrectly flag some headlines, CRTA's reports should remain the main source of truth.

The main users of this service would be watchdog organisations in Serbia and foreign policymakers. Since Serbia is due to hold presidential elections this year due to Vučić's resignation, the project could also be used to monitor election integrity during the campaign period.

Data collection

The headlines for training machine learning models were collected by scraping the politics sections of Kurir and Informer portals. Both web pages and RSS feeds were scraped. The data was labelled using Claude Sonnet 5.5, and all labels were verified by a Serbian speaker. A small number of labelled example headlines were collected from CRTA's website.

The dataset is stored as a CSV file and contains approximately 1300 headlines. The file contains the following columns: headline, label, source (outlet), url and date.

Preprocessing

A mixture of the Latin and Cyrillic scripts was used in the headlines. All headlines were first normalised to Latin, and diacritic folding was applied. Next, stopwords were removed. However, stopword removal was skipped for the model based on Sentence Transformer embeddings. Finally, the headlines were deduplicated.

Exploratory Data Analysis (EDA)

We didn't receive a data set from CRTA in time for the project, but just from the headlines we labeled ourselves, we could see that there was at least some coverage of all of the labels, and also the share of "nothing" labels wasn't too high: this meant that the categories selected *a priori* do in fact represent over 80% of the data.

Table 1: Dataset breakdown by label

label	count	%
nothing	84	18.5
personality_cult	38	8.4
vilifying_eu	19	4.2
vilifying_neighbours	69	15.2
vilifying_opponents	245	53.8

This meant we didn't need to add more categories but could safely just use those that we planned. Of course since they came from CRTA, we knew they were "good" in the sense that they were extensively battle-tested in manual processes. What we were unsure about was if they'd be representative of shorter time frame news-feed: some of the labels could be very seasonal or otherwise rarely used just when we would be sampling.

Seasonality is in fact visible in the data: since the election is coming, the propaganda apparatus is in full gear smearing the political opposition, leaving less bandwidth for vilifying the European Union.

But the EDA resulted us being very much satisfied with our current plan and current data collection methods. So: on to analyzing deeper what we got, and building predictive models!

Modelling

We applied all tools introduced during the course to this classification task. Good thing is, this is very textbook "sentiment analysis": given sentences, we are to classify them into 5 buckets. For encoding the headlines into embedding vectors, we used TF-IDF vectorizers and transformers. For classifying the vectors into label classes, we tried logistic regression, decision trees, and an "automated machine learning" library called Tree-based Pipeline Optimization Tool (TPOT), introduced in week 6 exercises.

Results

Table 2: Performance metrics for different models

model	accuracy	precision	recall	f1_score
Transformer + TPOT search	0.788	0.670	0.738	0.696
Transformer + logistic regression	0.826	0.628	0.728	0.666
TF-IDF + logistic regression	0.742	0.571	0.568	0.557
TF-IDF + TPOT search	0.667	0.554	0.560	0.528
Hierarchical logistic regression	0.697	0.547	0.484	0.493
Decision tree	0.614	0.384	0.356	0.355
Majority baseline	0.538	0.108	0.200	0.140

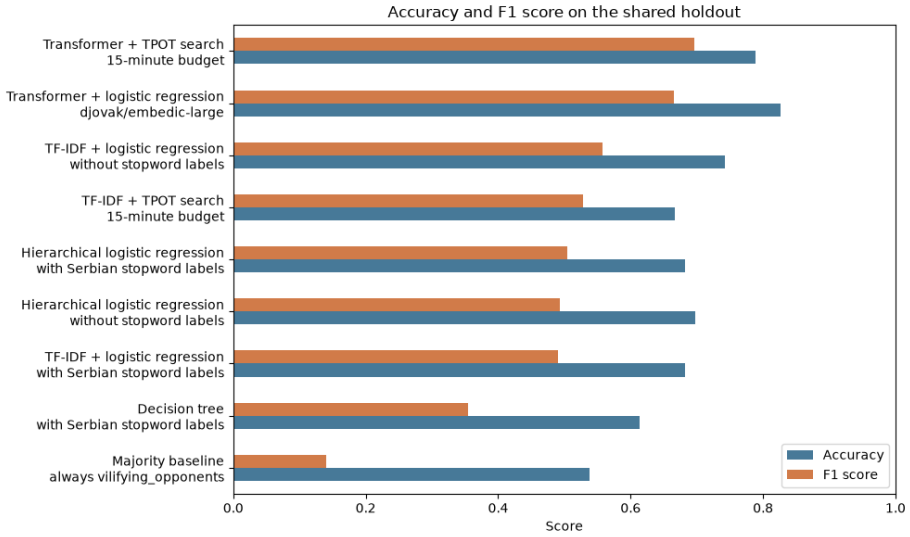


Figure 1: Diagram

There were two kinds of classification ideas: logistic regression and decision trees. TPOT would also produce a pipeline that ended with `SGDClassifier`, a linear classifier like logistic regression with Stochastic Gradient Descent training. Curiously, it would lose to simple `LogisticRegression` with TF-IDF and just barely inch ahead when using the `SentenceTransformer`.

There is certain tradeoff between good scores and interpretability: the most interpretable methods are decision trees and logistic regression with TF-IDF, but the best scores are achieved by using SentenceTransformer embeddings that are opaque to the human eye.

Next I'll detail each technique and the results obtained.

TF-IDF

Term frequency minus inverse document frequency (TF–IDF) is a technique that was introduced in week 2 (MOOC chapter 4) exercises. Quoting Wikipedia, TF minus IDF is "a measure of importance of a word to a document in a collection or corpus, adjusted for the fact that some words appear more frequently in general". Here "document" means simply the headlines we're classifying.

Term frequency means more common words within the document are more important (locally), while **inverse** document frequency means more globally common words between ALL documents are less important. This makes sense: some small words are used everywhere, so they can help very little telling the different headline classes apart. Of course the very most common words are filtered out completely during so-called stopword filtering, already introduced in the week 1 (chapter 3) exercises.

The good thing about TF-IDF feature vectors is that the words themselves stay as-is, so whatever weights they receive are fully interpretable, even after logistic regression or decision tree modelling.

Sentence transformer

Transformers were introduced in chapter 5 and the week 4 exercises. The idea to use them for improved embeddings actually came from the audience feedback during the spotlight session Q&A, so a big thank you to our active audience!

While the transformer produces much richer 1024-dimensional vectors than the simple TF-IDF score, those vectors are also quite impossible to interpret: best we can do is look how our labelled classes are spread in the embedding space:

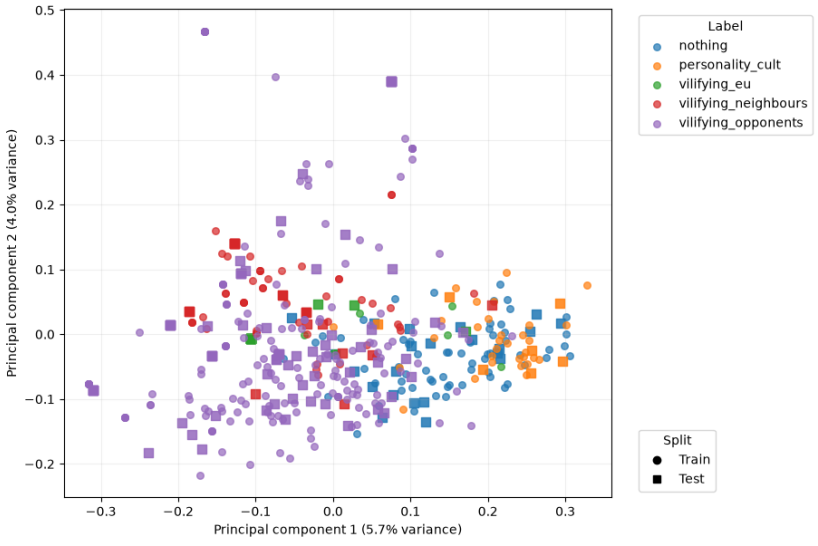


Figure 2: 2D projection of the transformer embeddings

And it indeed looks very good! Except for the `vilifying_eu` label, the other

classes seem neatly linearly separable; indeed, simple linear regression gets them very nicely separated:

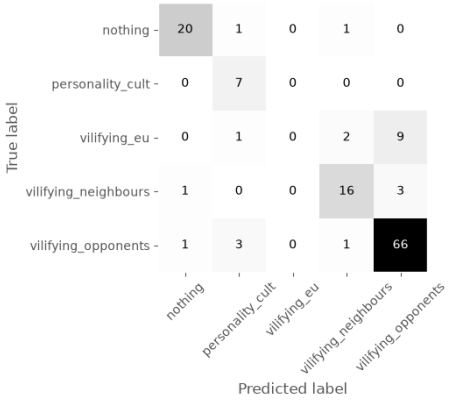


Figure 3: Confusion matrix for transformer embeddings + logistic regression

Decision trees

Decision trees were introduced in chapter 5. The scores for the decision tree model are not the best, but its advantage is interpretability. At the top of the tree we already see two common words: Vučić, and blokaderi (a derogatory term for political opposition and demonstrators).

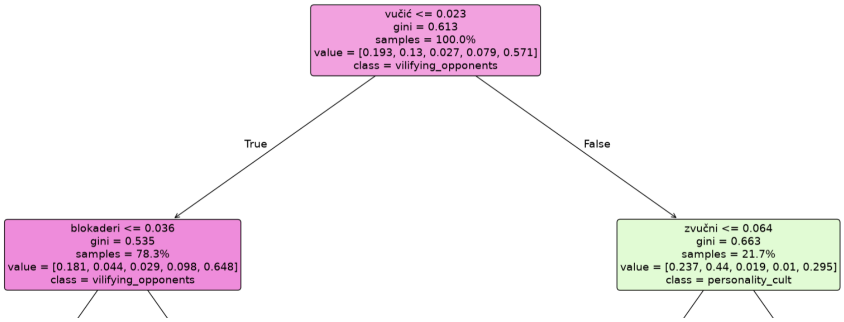


Figure 4: Top of the decision tree

The app

We present the headline classification results in a web app inspired by CRTA's monthly media manipulation reports. Users can specify a time period and check the number of headlines in each category.

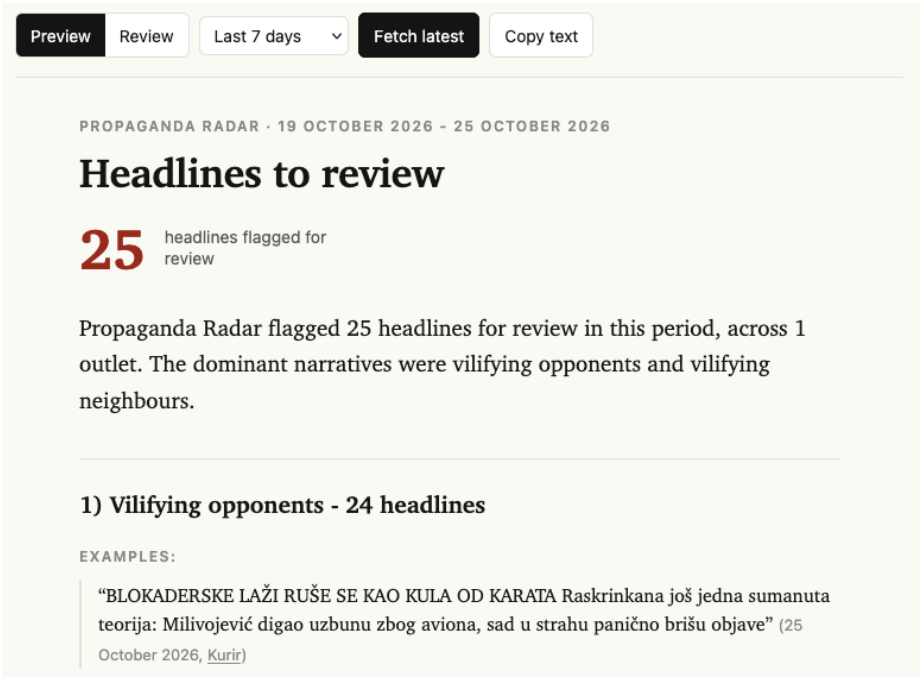


Figure 5: Distilled results

Labellers can verify AI-labelled headlines. We plan to add support for posting blog posts to the platform. Keywords important to determining a headline's category are highlighted.

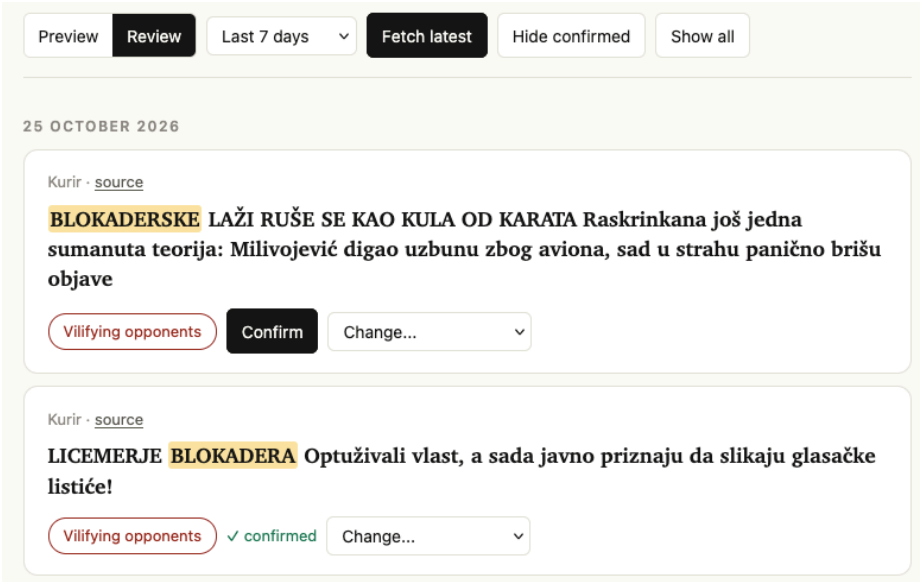


Figure 6: Efficient manual review

The frontend is built with Alpine.js, a lightweight JavaScript framework that enables dynamic updates without a build step. The app reads the data store live, providing real-time updates to the user interface. The backend is implemented using Python's built-in http package to minimise complexity.

Conclusion

We have delivered an end-to-end data science project covering all stages of the data lifecycle. The data science project canvas taught us how to plan a data project. Scraping multiple websites taught us how to organise data from different sources. During the preprocessing and modelling stage, we got to apply all textual data analysis tools taught during the course. Finally, preparing the spotlight presentation enhanced our ability to convey our ideas and show the value of our work to stakeholders efficiently.

Reflecting on the process: the initial plan held very well. Everything in data collection and analysis went very smoothly, despite us not succeeding in contacting CRTA and getting their hand-labelled datasets. This is understandable: they had more important things to do during the election season. The incumbent president just admitted defeat, so maybe they did their part right!

We hope the tool can be useful to political observers, later elsewhere, since it should be straightforward to adapt to other countries and other pivotal times.

References

[1] Media Pluralism Monitor 2026, https://cmpf.eui.eu/country/serbia-2026/

[2] CRTA Media and Information Space Page, https://crta.plus/media-information-space/

[3] CRTA Front Page Manipulations Overview, https://crta.plus/media-information-space/front-page-manipulations/

APPENDIX: PROJECT CANVAS

Title: Propaganda radar
Group members: Momir, Viljami, Juuso

Workshop # : 3

MOTIVATION

customer: CRTA is an organization that want to promote transparency in state media influence

end-user: probably someone who either enters the site via link (from social media or similar), so probably critical of the state influence; or maybe anyone who has interest in the relationship between media, the government and politics

benefit to CRTA: more effective reporting (real-time, automated)

benefit to end-user: politologists or other researchers get more data, to support a bigger picture; casual visitor gets a more interactive and real-time experience

DATA COLLECTION

Selected government-controlled media sources: headlines, articles from their web sites. Via RSS, then scrape with site-specific custom scrapers

Data management plan:

Scraped data will live in the analysis backend, not exposed

from the output of the analysis, render dashboards (classification of news items) etc.

For test and training data, we'll ask CRTA for a hand-labelled data set

PREPROCESSING

Scraped data needs to be cleaned up for the analysis.

Goal is to classify the headlines (later also articles) into pre-specified buckets (classes like "personality cult building"). Initial buckets come from CRTA's article. Based on EDA we might need to add more buckets.

Apply common text cleaning and simplification steps (stemming etc.), learn to associate keywords and phrases to the classes.

EXPLORATORY DATA ANALYSIS (EDA)

For "live feed", or classifying a random article, it will be important to evaluate how good the coverage of the buckets is.

Figure out the rough proportion of each class by random sampling from the web site.

This has implications to training: if there's a lot of items that are outside the buckets, we need to collect more "random samples" ourselves, perhaps add more buckets

VISUALIZATIONS

Visualize the feature vector outputs by the classifier (vs hand-labelled ground truth), maybe some kind of PCA plot vs labels

Word clouds corresponding to each bucket

Clustering that would support or reject the current buckets (do we need more? Are some redundant or empty?), after preprocessing with BERT

LEARNING TASK (focus on problem definition)

Problem: Classifying pieces of text to pre-defined classes

Supervised learning: learn classes from hand-labelled data

Input: headline (later a full article)

Output: vector of probabilities of belonging to each class

Learning target: highest probability corresponds to the correct label

LEARNING APPROACH (focus on solution implementation)

Classification: Logistic Regression, Random Forest, XGBoost, SVM

Evaluation metric: F1 score

Hierarchical classification: is it any of the buckets? Vilifying vs glorifying? Consider training separate classifiers to each step.

For training data: hand-labeled data from CRTA + own scraping

COMMUNICATION OF RESULTS

Classifications of the past articles: lists of the classification results (buckets + the corresponding headlines)

Live feed (chronological list) of the news items after they've been analyzed: classification (later also the specific keywords or phrases that support the classification)

DATA PRIVACY AND ETHICAL CONSIDERATIONS (if applicable)

The classification buckets can be very negative: we should present the classification as a "weak claim" until it has been verified: either manually by a trusted moderator, or by the community. There must be a human in the loop.

All scraped raw text is public (no need for explicit consent), and we won't re-distribute it. Headlines can't be copyrighted.

There's no PII that wasn't included in the public text (no anonymization needed).

We don't collect data from the users.

ADDED VALUE

Politologists or other researchers get more data, to support a bigger picture

A casual visitor gets a more interactive and real-time experience

LEGEND

WEEK 1: Data collection/preprocessing

WEEK 2: EDA & visualizations

WEEKS 3-4: Machine/deep learning